

Machine Learning and Agentic AI for Macroeconomic Research

7: Large Language Models & Text as Economic Data

Zhigang Feng

Spring 2026

Lecture Agenda

1. **Why LLMs for Economists?** — motivation, history, the autoregressive principle
2. **Tokenization** — mapping raw text to integers
3. **Word Embeddings** — TF-IDF, Word2Vec, GloVe
4. **The Transformer** — a function on sequences of vectors
5. **Self-Attention** — the heart of the Transformer
6. **A Full Layer** — residual, norm, MLP, multi-head
7. **Generation, Pre-training, Fine-tuning** — how the brain is built and adapted
8. **Economic Applications** — three case studies, application zoo, and 2024–25 frontier
9. **Limitations** — knowledge cutoff, hallucination, prompt sensitivity
10. **Transition to RAG** — from a static brain to a live, knowledge-augmented system (Lec08)

Why LLMs? Motivation and History

Why Economists Care About LLMs

- **Text as data** — modern economic research increasingly leverages textual sources:
 - Corporate filings (10-K, 10-Q, MD&A)
 - Central bank speeches, FOMC minutes, ECB statements
 - Earnings calls, analyst reports, regulatory filings
 - News, social media, congressional records
- **Applications:**
 - Sentiment analysis for financial forecasting
 - Policy stance detection and uncertainty indices
 - New text-derived macroeconomic indicators
 - Summarization of long, technical documents
- **Why LLMs in particular:**
 - Capture nuanced language and domain-specific jargon
 - Handle long-range context (sentences, paragraphs, whole documents)
 - Provide a unified architecture for many tasks

Three Roles of Text in Economic Research

Role	What text is	Example
Measurement tool	A way to construct previously unobserved variables	Baker–Bloom–Davis (2016) Economic Policy Uncertainty index from newspapers
Behavioral outcome	A response variable revealing decisions or institutional change	Hansen, McMahon & Prat (2018): how FOMC transparency reform changed committee speech
Treatment variable	A quasi-experimental input affecting behavior	Central bank communication shocks and inflation expectations

Take-away: the same text can play very different econometric roles. Choose your representation method to match the role—transparency matters for measurement, while subtle context matters for behavior modeling.

A Brief History: RNN → Transformer → GPT

- **2003:** Bengio et al. neural language model — continuous word vectors
- **2013:** Mikolov et al. *Word2Vec* — dense word embeddings at scale
- **2014:** Pennington et al. *GloVe* — global co-occurrence matrices
- **2017:** Vaswani et al. “*Attention Is All You Need*” — the Transformer replaces RNNs/LSTMs
- **2018:** Devlin et al. *BERT* (encoder); Radford et al. *GPT* (decoder)
- **2020–2022:** GPT-3, instruction tuning, RLHF → ChatGPT moment
- **2023–2026:** GPT-4/4o, Claude (3, 4), Gemini, LLaMA-3, DeepSeek; long-context (100k–1M tokens), open-weights ecosystem, agentic systems

Driver of the shift: attention scales better than recurrence, both in compute (parallel) and in quality (long-range dependencies).

The Autoregressive Principle

- An LLM is fundamentally a **next-token predictor**:

$$p(t^{(n+1)} \mid t^{(1)}, t^{(2)}, \dots, t^{(n)}).$$

- **Generation:** sample $t^{(n+1)}$ from this distribution, append to the context, and repeat.
- **What this single equation enables** (we preview the intuition on the next two slides; a full revisit comes after we understand pretraining and fine-tuning):
 - **Summarizing 10-K filings or FOMC minutes** — prompt with document + instruction; the model predicts summary tokens one by one
 - **Generating narrative forecasts** conditional on macro state — condition on data in the prompt; next tokens extend the narrative
 - **Producing scenario analyses in plain language** — the scenario is the context; prediction fills in the implications
 - **Writing and debugging code** for empirical work — same objective over a code corpus yields code completions
- **Key insight:** “everything is next-token prediction” is mathematically tiny but pedagogically and practically enormous.

Intuition: Summarizing a 10-K Filing

- **Task:** condense 50 pages of an SEC 10-K risk section into 3 readable bullets.
- **How the autoregressive equation achieves this:**
 1. Concatenate the instruction and document into the prompt:
■ Summarize the following 10-K risk section in 3 bullets:\n\n[text]\n\nSummary:■
 2. The model predicts token $t^{(n+1)}$ from the full prompt; this becomes the first word of the summary
 3. Each subsequent token is predicted given everything before it (including the already-generated summary words)
 4. Generation stops at a period or end-of-sequence marker
- **Why does it work?** During pretraining the model read millions of (long document, short summary) pairs. The next-token distribution has implicitly learned: *“after an instruction like this, produce a concise factual summary.”*
- **Economic punchline:** a 50-page MD&A section is compressed into 3 bullets in under one second — and the same pipeline scales across thousands of filings.

Intuition: Extracting Stance from FOMC Minutes

- **Task:** classify each sentence in FOMC minutes as *hawkish*, *dovish*, or *neutral*.
- **How the autoregressive equation achieves this:**
 1. Prompt: ■Sentence: [sentence]\nStance (hawkish/dovish/neutral):■
 2. The model predicts the next token; because the vocabulary contains these exact words, the top-probability token *is* the label
 3. With fine-tuning on labeled FOMC data, the next-token distribution concentrates on monetary-policy-relevant labels
- **What fine-tuning changes:** the base model already understands “hawkish” and “dovish”. Fine-tuning shifts the distribution so that, after a central-bank sentence, stance-label tokens are consistently most probable.
- **Economic application:** automated stance extraction across decades of FOMC minutes — enabling the kind of large-scale communication analysis in Hansen, McMahon & Prat (2018) without manual coding.

An Identification-First View of Text

- **Engineering perspective first:** to a machine, a document is just a character string. But every string was *produced* — by a person, in a situation, for an audience. Before any econometrics, ask: what generated this string?

- **The data-generating process (DGP):**

$$x_i = g(s_i, a_i, c_i) + \varepsilon_i$$

- s_i : the latent **economic state** we want to measure (policy stance, sentiment, firm risk)
- a_i : the speaker's **strategic framing** — audience management, spin, emphasis, tone chosen for effect
- c_i : **institutional context** — venue, legal disclosure requirements, format conventions
- **Why this matters for a downstream regression** $y_i = \beta m(x_i) + \gamma z_i + u_i$:
 - If framing a_i is correlated with the error u_i , the text feature $m(x_i)$ is *endogenous* — the same omitted-variable problem as always, but hidden inside the language
 - Measurement error is non-classical: it correlates with the true state s_i , biasing $\hat{\beta}$
- **Two research uses:** (a) **Measuring** s_i — choose representations that filter out a_i ; (b) **Studying strategic communication** — a_i is your outcome variable (e.g. Hansen, McMahon & Prat 2018).

Tokenization

Tokenization: The First Step

- **Goal:** convert raw character sequences into discrete *tokens* that a neural network can consume.
- **Why not just split by spaces?**
 - Many economically relevant terms combine words, numbers, and symbols (e.g. S&P500, CO2, Q4_2025, stagflation)
 - Punctuation and formatting carry meaning ('\$1.2B' vs '1.2 billion')
 - Different languages have different word boundaries (or none at all)
- **What is a token?** A flexible unit:
 - A whole word ('inflation')
 - A subword ('infl', 'ation')
 - A single character or byte
 - A special marker (<EOS>, <PAD>, <USER>)

Vocabulary and the Tokenization Map

- Let A be the alphabet (Unicode characters). A tokenizer is a function:

$$\mathcal{K}: A^* \longrightarrow \{1, 2, \dots, M\}^*$$

mapping any text string into a sequence of **numbers** — each number is a positive integer that serves as an index into a lookup table of size M . The fact that the index is an integer is incidental; what matters is that it *uniquely identifies* each token, the same way a ZIP code uniquely identifies a location.

- **Why integers?** Neural networks need numeric inputs; each token ID will index into an embedding lookup table.
- **Practical constraint — context length:**
 - Models have a maximum input size n (typically 2,048 to 128,000 tokens; some 1M)
 - Long policy documents, FOMC transcripts, or 10-K filings often exceed n
 - Forces *chunking*, *summarization*, or *retrieval* (a hint at Lec08)
- Vocabulary sizes are typically 30,000–100,000 subword tokens. **To put this in perspective:** Merriam-Webster Unabridged lists $\approx 470,000$ entries; a college-educated native speaker actively uses roughly 20,000–40,000 words. A subword vocabulary of $M = 50,000$ therefore spans roughly $\frac{470,000}{50,000} \approx 9.4$ words.

Illustrative Example: “GDP rose by 2%”

- A simple tokenization might produce

(GDP, rose, by, 2, %)

mapping to integer IDs, e.g. (541, 823, 71, 312, 49).

- **Why integer mapping matters:**

- Each ID becomes the index of a learned embedding vector
- Numeric tokens ('2', '3', '5') get *their own* embeddings

- **Injectivity matters for forecasting:** a well-designed tokenizer maps

- “inflation rate is 3.5%”
- “inflation rate is 3.6%”

to *different* token sequences. Otherwise small but economically meaningful differences would collapse into the same input.

- **Take-away:** tokenization is the silent foundation. Bad tokenization $\Rightarrow$ everything downstream is fighting noise.

Subword Tokenization (BPE / WordPiece)

- Modern LLMs use **subword** tokenizers, learned from a large corpus by greedy frequency merging:
 - Byte-Pair Encoding (BPE): GPT, LLaMA
 - WordPiece: BERT
 - SentencePiece: T5, multilingual models
- **Why subwords?**
 - Handle out-of-vocabulary words (`QuantitativeEasing` → `Quant+itative+Easing`)
 - Compress common words to one token; split rare or compound words
 - Avoid the explosion of a full-word vocabulary
 - Work across languages with the same vocabulary
- **Cost:** a previously coined term like `nowcasting` may split into 3 tokens; a numeric figure like `1.2345` may split into several digit-tokens, making numerical reasoning fragile.

Tokenization Is a Preprocessing Choice

- Tokenization sits inside a larger **preprocessing pipeline**:

$$x^{\text{raw}} \xrightarrow{\text{clean}} x^{\text{clean}} \xrightarrow{\text{tokenize}} \{t_{ij}\} \xrightarrow{\text{represent}} z_i,$$

from raw scraped text $\rightarrow$ normalized text $\rightarrow$ integer token IDs $\rightarrow$ downstream numeric representation (TF-IDF, embedding, ...).

- **Specification curve problem** (Denny & Spirling, 2018): just as economists report results under alternative lag orders, control sets, and sample restrictions, NLP results should be reported under alternative preprocessing pipelines.
 - LDA topics under different pipelines vary substantially — often by as much as the substantive variation of interest
 - Stemming and punctuation handling have the largest individual effects
 - **Lesson:** the pipeline is a modeling decision; report the full range as you would a VAR robustness table
- **Lesson for empirical economists:** report the *universe* of preprocessing choices. The “convenient” specification is not a result — it is a choice. This applies equally to tokenizer choice and vocabulary size.

The Seven Preprocessing Choices

- **Lowercasing** — map Fed and fed to the same token. Compresses vocabulary, but erases proper-noun signal.
- **Stemming / lemmatization** — cut suffixes so *banking/banks/banked* collapse to *bank*. Can erase tense, plurality, policy-language nuance.
- **Stop-word removal** — drop high-frequency function words. Reduces noise; but negation (*not*) and modality (*may, will*) can matter for central-bank text.
- **Punctuation handling** — keep, strip, or tokenize separately. Percent signs, decimals, sentence boundaries all matter.
- ***n*-gram inclusion** — treat “*yield curve*” as one token rather than two. Captures multi-word economic concepts at the cost of a larger vocabulary.
- **Frequency cutoffs** — drop terms in $< k$ or $> K$ documents. Removes typos and boilerplate; can also remove rare-but-informative events.
- **Language filtering** — restrict to English or apply per-language pipelines. Crucial for multilingual corpora like ECB speeches.

Each choice is an independent on/off switch $\Rightarrow 2^7 = 128$ pipelines $\Rightarrow$ report a *specification curve*, not a single run.

Word Embeddings

Why Embed Tokens into Vectors?

- Neural networks operate on real-valued vectors, not discrete symbols.
- An **embedding** is a learned function

$$\mathcal{E}: \{1, \dots, M\} \longrightarrow \mathbb{R}^d,$$

mapping each token ID to a dense vector of dimension d (typically $d = 128$ to 4096).

- **Think of it as a basis problem from linear algebra.** We want to find a coordinate system — a set of d orthogonal “semantic axes” (e.g. monetary-policy stance, positive valence, corporate context) — such that every token maps to a point in $\mathbb{R}^d$. Once tokens are coordinates, we can do arithmetic: measure distances, compute cosine similarities, add and subtract meaning. We will later connect this to Arrow–Debreu securities in financial markets. The challenge: we do not know the right axes in advance. They must be *learned* from data.
- **What we want from embeddings:**
 - Semantically similar tokens should have nearby vectors (close coordinates in the learned basis)
 - Algebraic operations (sums, differences) should reflect semantic relations
 - The mapping should be *learned* from data, not hand-designed
- **Bonus:** we will also need a *positional* signal so the model knows token order—more on this when

Three Approaches to Word Representation

The problem from the previous slide: find a mapping from tokens to vectors that captures semantic relationships. Three generations of methods have tried to solve this, each fixing a specific failure of its predecessor. Understanding this progression explains *why* the Transformer was needed.

Method	Type	Idea	Year
TF-IDF	Sparse, frequency	Re-weight word counts	1970s
Word2Vec	Dense, predictive	Predict neighbors in window	2013
GloVe	Dense, statistical	Factor co-occurrence matrix	2014
Transformer embeddings	Dense, contextual	Vary by surrounding tokens	2017+

- TF-IDF: a bag-of-words feature. Transparent and easy to interpret.
- Word2Vec / GloVe: produce a single vector per word *type* (*static* embeddings).
- Transformer embeddings: produce a different vector for each *occurrence* of a word, depending on its surrounding context (*contextual* embeddings).

Term Frequency (TF)

- **Definition:** share of document d consisting of word w :

$$\text{TF}(w, d) = \frac{\text{count}(w, d)}{\text{total words in } d}.$$

- **Toy example.** FOMC-like sentence d : “*the Fed raised rates because inflation persisted*” (7 words). Every word appears once, so TF is $1/7 \approx 0.14$ for each:

word	count in d	$\text{TF}(w, d)$
<i>the</i>	1	0.14
<i>Fed</i>	1	0.14
<i>raised</i>	1	0.14
<i>rates</i>	1	0.14
<i>inflation</i>	1	0.14

- **Limitation:** TF rewards *frequent* words. Across a corpus, function words (*the, a, is*) dominate even though they carry almost no semantic signal. We need a counterweight.

Inverse Document Frequency (IDF)

- **Definition:** how rare word w is across a collection of N documents:

$$\text{IDF}(w) = \log\left(\frac{N}{\#\{\text{docs containing } w\}}\right).$$

- **Toy example.** Corpus of $N = 100$ FOMC speeches:

word	docs containing it	IDF(w)
<i>the</i>	100	$\log(100/100) = 0$
<i>inflation</i>	60	$\log(100/60) \approx 0.51$
<i>yield curve</i>	5	$\log(100/5) \approx 3.00$

- **Intuition:** rare $\Rightarrow$ informative $\Rightarrow$ up-weighted. “*the*” gets 0 automatically — no stop-word list needed. “*yield curve*” gets a large multiplier because it distinguishes a few speeches from the rest.

Putting It Together: TF-IDF

- **Product formula:** each (word, document) pair gets a weighted score

$$\text{TF-IDF}(w, d) = \text{TF}(w, d) \cdot \text{IDF}(w).$$

Stacking across all words in vocabulary $V \Rightarrow$ each document is a sparse vector in $\mathbb{R}^{|V|}$.

- **Toy 3-speech matrix** (illustrative scores):

	<i>the</i>	<i>inflation</i>	<i>yield curve</i>
speech A (hawkish)	0	0.08	0.00
speech B (dovish)	0	0.05	0.00
speech C (technical)	0	0.02	0.21

- **Properties:**

- Down-weights common words automatically (no stop-word list)
- Highlights distinctive words for each document
- Lives in $\mathbb{R}^{|V|}$ — very high-dimensional, very sparse
- *No semantic similarity: “central bank” and “Federal Reserve” are orthogonal $\Rightarrow$ motivation for dense embeddings (next slide).*

Word2Vec: Dense Embeddings from Context

- **Distributional hypothesis** (Firth, 1957): “*You shall know a word by the company it keeps.*”
- **Idea** (Mikolov et al., 2013): for each word in a corpus, look at a window of $\pm L$ neighboring words. Learn a vector for the center word that *predicts* its neighbors (*Skip-gram*) or learn one that *is predicted by* its neighbors (*CBOW*).
- **Training output:** a single static vector $\mathbf{u}_w \in \mathbb{R}^d$ for each word w in the vocabulary, with d typically 100–300.
- **Geometric magic:** after training, simple vector arithmetic captures analogies, e.g.

$$\mathbf{u}_{\text{king}} - \mathbf{u}_{\text{man}} + \mathbf{u}_{\text{woman}} \approx \mathbf{u}_{\text{queen}}.$$

Words living on the same semantic axis differ by approximately the same vector.

Skip-Gram in One Slide: The Math

- Each word w has *two* learned vectors: an input vector $\mathbf{u}_w$ and an output vector $\mathbf{v}_w$ (often merged after training).
- For a center word w_t and a context word w_c in its window, the model predicts

$$P(w_c | w_t) = \frac{\exp(\mathbf{u}_{w_t}^\top \mathbf{v}_{w_c})}{\sum_{j=1}^M \exp(\mathbf{u}_{w_t}^\top \mathbf{v}_{w_j})}.$$

- Training objective (negative log-likelihood over all (center, context) pairs):

$$\mathcal{L} = - \sum_t \sum_{c \in \text{window}(t)} \log P(w_c | w_t).$$

- Optimized with stochastic gradient descent (in practice with *negative sampling* to avoid the full vocabulary softmax).
- **Result:** words with similar context distributions end up with similar $\mathbf{u}$ vectors.

Skip-Gram: Walkthrough on “central banks target inflation”

- **Sentence:** “*central banks target inflation*”, window radius $L = 2$, center word = *target*.
- Training pairs formed from this center word:

$(target, central), (target, banks), (target, inflation).$

- For each pair, the score is the dot product $\mathbf{u}_{target}^T \mathbf{v}_{context}$. Gradient descent *increases* this score for observed pairs and *decreases* it for random (negative) pairs.
- **Across the whole corpus:** *target* appears near *rates, inflation, policy, Fed*. So do *raise* and *cut*. Consequently

$\mathbf{u}_{target}$ ends up close to $\mathbf{u}_{raise}$, $\mathbf{u}_{cut}$ and far from $\mathbf{u}_{apple}$.

- **Takeaway:** analogies emerge because consistent co-occurrences place related words at similar positions in $\mathbb{R}^d$.

GloVe: Global Co-occurrence Statistics

- Pennington et al. (2014): instead of sliding-window prediction, factor a *global* matrix.
- Let X_{ij} count how often word j appears near word i across the entire corpus. GloVe fits embeddings to satisfy

$$\mathbf{u}_i^\top \mathbf{v}_j + b_i + b_j \approx \log X_{ij}.$$

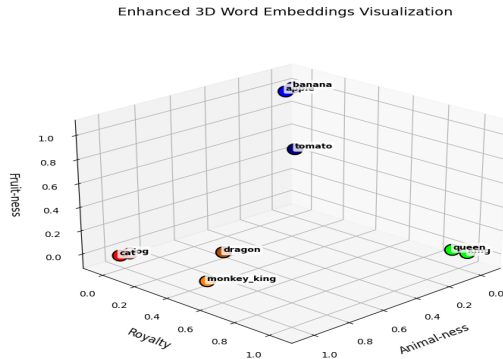
- **Toy counts on a central-bank corpus** X_{ij} (illustrative):

	<i>rates</i>	<i>inflation</i>	<i>apple</i>
Fed	820	410	3
ECB	540	390	2
hawkish	210	160	0

$\log X_{\text{Fed}, \text{rates}} \approx 6.7$ (large target dot-product) vs. $\log X_{\text{Fed}, \text{apple}} \approx 1.1$ (small). Fitting the equation therefore forces $\mathbf{u}_{\text{Fed}}$ close to $\mathbf{v}_{\text{rates}}, \mathbf{v}_{\text{inflation}}$ and orthogonal to $\mathbf{v}_{\text{apple}}$ — monetary-policy structure falls out of corpus-wide statistics.

- **Contrast with Word2Vec:** predictive / local / streaming (W2V) vs. statistical / global / one-shot factorization (GloVe). Both produce *static* dense vectors with similar geometry. Pick W2V for massive streaming corpora. GloVe when a full co-occurrence matrix is cheap to build^{24 / 78}

A 3D Toy of Word Embeddings



The three axes loosely correspond to “Animal-ness”, “Royalty”, and “Fruit-ness”. Words like *cat*, *king*, and *apple* cluster along the dimension that fits them best. Real LLM embeddings live in hundreds of dimensions, but the same intuition holds.

Analogy: Embeddings as Arrow–Debreu Securities

- For an economist, a d -dimensional embedding is reminiscent of an asset's payoff vector across d states of the world.
- **Each dimension** = a latent “semantic state” (royalty, animal-ness, monetary-policy stance, ...).
- **Each word** = a vector of exposures to those states.
- **Dot product** = a kind of correlation:
 - High dot product $\Rightarrow$ similar payoff structure $\Rightarrow$ similar meaning
 - Orthogonal vectors $\Rightarrow$ unrelated concepts
- **Cosine similarity** normalizes for vector length:

$$\cos(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u}^\top \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|}.$$

- This view makes embeddings feel like a familiar object: a state-contingent payoff vector in a learned semantic state space.

Limits of Static Embeddings

What static embeddings miss

- **Polysemy:** “rate” = interest rate? exchange rate? growth rate? One vector for all senses.
- **Negation:** “GDP growth NOT slowing” and “GDP growth slowing” have nearly identical mean vectors.
- **Long-range context:** fixed window misses dependencies across paragraphs.
- **Corpus dependence:** “mask” before vs. after 2020 are very different concepts.

What we need next

- Vectors that change with *context*, not just word identity
- A mechanism for tokens to “look at” other tokens, near or far
- A way to learn arbitrary relational patterns, not just co-occurrence within a window

⇒ **The Transformer.**

Transformer High-Level

How the Transformer Works in Plain Language

- Consider the sentence: *“The Fed raised rates to combat inflation.”*
- When you read the word **rates**, you instinctively ask: *“rates of what? raised by whom? why?”* — your eye jumps back to *Fed* (subject) and forward to *inflation* (motivation).
- **A Transformer does exactly this, for every token simultaneously:**
 1. Every token asks: *“which other tokens are most relevant to understanding me?”*
 2. Each token collects a weighted blend of information from every other token in the sequence
 3. All positions are processed in **parallel** — not one-by-one as in an RNN
- Repeat this “everyone looks at everyone” operation L times (e.g. $L = 96$ in GPT-3) with different learned weights, and the model builds deeply contextualized representations.
- **Engineering name for this operation:** scaled dot-product self-attention. The next slides unpack the five steps in detail.
- **Contrast with Word2Vec / GloVe.** Those models produced *one vector per word type*, independent of context — *bank* in “river bank” and *bank* in “central bank” share the same $\mathbf{u}$. A Transformer produces *one vector per token occurrence*, computed from its neighbors — the two *banks* end up at different points in $\mathbb{R}^d$. **Static** $\rightarrow$ **contextual** is the single conceptual jump.

The Transformer as a Function

- A Transformer is a function

$$\mathcal{T}: (\mathbb{R}^d)^n \longrightarrow (\mathbb{R}^d)^n, \quad [e^{(1)}, \dots, e^{(n)}] \mapsto [e'^{(1)}, \dots, e'^{(n)}]$$

mapping n input vectors to n output vectors of the same dimension. Inputs are token embeddings plus positional encodings; outputs are *contextualized* embeddings.

- **What “enriched” means.** Each $e'^{(i)} \in \mathbb{R}^d$ encodes at once (a) the identity of token $t^{(i)}$, (b) its grammatical/semantic role in *this* sequence, (c) a weighted blend of every other token's contribution (causally masked in autoregressive models).
- **Important:** these are the Transformer's direct output — an *intermediate representation*, **not the final answer**. Inside: L stacked layers, each containing self-attention and a feed-forward network.
- **Turning embeddings into task output.** A small *task head* sits on top:
 - Generation: project $e'^{(n)}$ onto the vocabulary $\rightarrow$ softmax $\rightarrow$ sample a token (Part VII).
 - Classification: project $e'^{([\text{CLS}])}$ onto class labels $\rightarrow$ softmax $\rightarrow$ pick a class.

Positional Encodings

- **Problem:** self-attention is permutation-invariant. The tokens $\{The, Fed, raised, rates\}$ give the same internal state whether we read “*The Fed raised rates*” or the scrambled “*Rates raised the Fed*” — yet the economic meaning is opposite (agent and patient are reversed). We must inject position.

- **Solution:** add a position vector $p(i)$ to each token embedding:

$$e^{(i)} = \mathcal{E}(t^{(i)}) + p(i).$$

- **Two flavors:**
 - **Sinusoidal** (Vaswani et al. 2017): fixed sine and cosine functions of position at multiple frequencies. Generalizes to longer sequences in theory.
 - **Learned:** positions $1, 2, \dots, n_{\max}$ get their own learnable vectors. Used in early GPT.
 - Modern variants: *RoPE*, *ALiBi* – bake position directly into attention scores; better extrapolation.
- **Take-away:** a Transformer must be *told* where each token is. The mechanism is simple but indispensable.

From Output Embeddings to the Next Token

- For autoregressive generation we focus on the *last* output vector $e'^{(n)}$.
- Project it onto the vocabulary with a linear layer:

$$\text{logits} = W_{\text{vocab}} e'^{(n)} + b_{\text{vocab}}, \quad W_{\text{vocab}} \in \mathbb{R}^{M \times d}.$$

- Convert logits to probabilities with softmax:

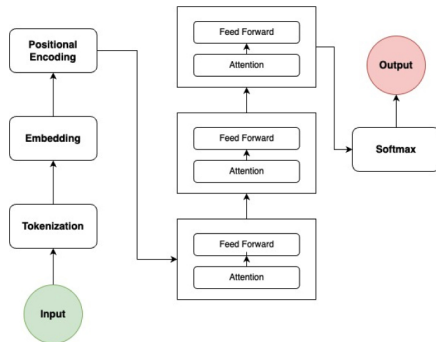
$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \frac{\exp(\text{logits}_w)}{\sum_j \exp(\text{logits}_j)}.$$

- **Sampling strategies:** greedy, top- k , top- p (nucleus), temperature scaling. (More on this in Part VII.)
- Append the chosen token to the context and repeat. **That** is how an LLM “generates text”.

Inside a Layer (Preview)

Each Transformer layer has four steps, each with a distinct job:

- **Self-attention** — *gathers context*: every token asks “who else in this sentence tells me what I mean?” and collects a weighted blend of their values.
- **Residual + LayerNorm** — *safety net*: the original vector is added back (so attention can only *add* information, never destroy it) and then normalized so deep stacks don’t blow up.
- **Feed-forward (MLP)** — *per-token transformation*: each token’s vector is passed through a two-layer non-linear network; this is where most of the model’s memorized knowledge lives.
- **Residual + LayerNorm** again — the same safety net after the MLP step.



Stack L such layers; the same shape $(\mathbb{R}^d)^n$ flows through each one.

Real LLM Scale

Quantity	Symbol	GPT-2	GPT-3	Modern (2024–26)
Layers	L	12–48	96	60–120
Hidden dim	d	768–1600	12288	4096–16384
Context length	n	1024	2048	32k–1M
Vocabulary	M	50k	50k	50k–256k
Parameters	—	0.1–1.5B	175B	70B–>1T

- **Modern open-weight examples (2024–2026):** LLaMA-3 (8B, 70B, 405B), DeepSeek-V3 (671B MoE), Qwen-2.5, Mistral.
- **Modern closed examples:** GPT-4/4o, Claude 3/4, Gemini 1.5/2 (sizes undisclosed).
- All share the same basic architecture; differences are scale, training data, and tuning.

Self-Attention Deep Dive

Self-Attention: The Key Idea

- For each position i in the sequence, ask: “*which other positions are most relevant to position i ?*”
- Compute a set of weights α_{ij} (with $\sum_j \alpha_{ij} = 1$) and form a new representation as a *weighted sum* of all positions.
- **Crucial properties:**
 - Every token can attend to every other token in one step (no recurrence)
 - Weights are computed dynamically from the input, not fixed
 - Operation is parallelizable across positions $\Rightarrow$ trains efficiently on GPUs
- This single operation replaces a tower of recurrent steps and is the central piece that made the Transformer work.

Step 1 — Project to Q, K, V

- For each input embedding $e^{(i)} \in \mathbb{R}^d$, compute three projections:

$$Q_i = W_Q e^{(i)} \quad (\text{query})$$

$$K_i = W_K e^{(i)} \quad (\text{key})$$

$$V_i = W_V e^{(i)} \quad (\text{value})$$

where $W_Q, W_K, W_V \in \mathbb{R}^{k \times d}$ are learned matrices and typically $k \leq d$.

- Mnemonic** (information retrieval):
 - Q_i is the *question* that token i asks: “what context do I need?”
 - K_j is the *description* that token j advertises: “here’s what I am about”
 - V_j is the *actual content* that j contributes if matched
- What W_Q, W_K, W_V are trained to achieve:** shape the projections so that the dot product $\langle Q_i, K_j \rangle$ is high when V_j ’s content is useful to what the task needs at position i , and low otherwise. The model discovers which pairings matter from the loss; it is never told in advance.
- Why three separate projections?** Separating roles enables *asymmetric* attention: *rates* can ask a question (Q) that matches *inflation*’s advertisement (K) without *inflation* having to ask the same question. The next slide shows what breaks if we drop or tie any of (Q, K, V) .

Why Three Projections? Counter-Examples

What goes wrong if we drop one of (Q, K, V) or tie them together?

- **Only Q (no K , no V).** There are no keys to match against and no values to deliver — “attention” reduces to a per-token linear map of $e^{(i)}$. No information is exchanged across positions.
- **Only Q and K (no V).** We can compute weights $\alpha_j^{(i)}$, but there is no content V_j to aggregate — the weighted sum has nothing to act on.
- **Only K and V (no Q).** Every token advertises and every token has content, but no one asks. The weights $\alpha_j^{(i)}$ are undefined — attention has no direction.
- **Tied $Q = K$.** A token matches most strongly to tokens like itself (query and advertisement point the same way). Directional lookups — *rates* $\rightarrow$ *inflation* without *inflation* asking anything of *rates* — collapse into self-similarity.
- **Tied $K = V$.** Advertisement and content share the same subspace — the model cannot advertise one feature (“I am a verb”) and deliver a different one (a tense signal or a sentiment).

Conclusion: attention needs all three roles kept separate — *asker*, *advertiser*, *provider*.

Step 2 — Scaled Dot-Product Scores

- For each pair (i, j) , measure the compatibility between query Q_i and key K_j by their dot product, scaled:

$$s_j^{(i)} = \frac{\langle Q_i, K_j \rangle}{\sqrt{k}}.$$

- Why divide by $\sqrt{k}$?** As the dimension k grows, dot products tend to grow too, pushing softmax into saturation (one term dominates, gradients vanish). Scaling by $\sqrt{k}$ keeps the variance of $s_j^{(i)}$ around 1.
- Search-engine analogy:**
 - Q_i is your search query
 - $\{K_j\}$ are document tags / metadata
 - Scores $s_j^{(i)}$ rank tags by relevance to the query
 - Top-ranked tags will let us read the corresponding “documents” V_j
- Why scale by $\sqrt{k}$?** As dimension k grows, the variance of $\langle Q_i, K_j \rangle$ scales with k . Large dot products saturate the softmax (one weight ≈ 1 , all others ≈ 0), making gradients vanish. Dividing by $\sqrt{k}$ keeps the variance ≈ 1 regardless of dimension.

Step 3 — Softmax → Attention Weights

- Convert scores into a probability distribution over positions:

$$\alpha_j^{(i)} = \frac{\exp(s_j^{(i)})}{\sum_{r=1}^n \exp(s_r^{(i)})}.$$

- Properties:
 - $\alpha_j^{(i)} \geq 0$ and $\sum_j \alpha_j^{(i)} = 1$
 - Concentrated weights $\Rightarrow$ “hard” attention to a few positions
 - Diffuse weights $\Rightarrow$ “soft” attention spread across many
- **Causal masking** (autoregressive models): when predicting position $i + 1$, set $s_j^{(i)} = -\infty$ for $j > i$ before the softmax. This prevents the model from peeking at the future during training.
- **Encoder models** (BERT) use bidirectional attention: every token sees every other.
- **Why softmax?** We want the output to be a *convex combination* of value vectors — weights that sum to 1, all non-negative. Softmax converts any real-valued score vector into exactly such a probability distribution, enabling an interpretable weighted average of content across the sequence.

Step 4 — Weighted Sum of Values

- Given the attention weights $\alpha_j^{(i)}$, the attention output for position i is the weighted sum of all value vectors:

$$\text{AttnVec}_i = \sum_{j=1}^n \alpha_j^{(i)} V_j \in \mathbb{R}^k.$$

- Interpretation:** token i “reads” content from all positions in the sequence, but pays most attention to those whose keys best match its query.
- Long-range, short-range, and same-position interactions all happen in the same step — contrast with RNNs where information has to propagate through many time steps.
- Note: V_j is what is being passed around. K_j only decides *how much* to pass; the content delivered is V_j .
- Why a weighted sum rather than a hard lookup?** Instead of selecting a single “most similar” token, the model retrieves a *blend*: for *interest* in a monetary-policy sentence, it mixes V_{rates} , $V_{\text{inflation}}$, and V_{raised} into one vector that reflects all three contexts simultaneously — richer than any single lookup could provide.

Step 5 — Project Back to Dimension d

- The attention output lives in $\mathbb{R}^k$, but we need a vector in $\mathbb{R}^d$ to feed the next layer. Project back with a learned matrix $W_O \in \mathbb{R}^{d \times k}$:

$$\tilde{e}^{(i)} = W_O \text{AttnVec}_i.$$

- This gives the contextualized representation of token i *after* self-attention.
- **Why project back to $\mathbb{R}^d$?** In multi-head attention each head operates in a smaller subspace $\mathbb{R}^k$ (with $k = d/H$) to keep compute manageable. Projecting back with W_O after concatenating heads restores the common dimension d , so the result flows into the next layer without any shape change.
- **Five-step recap:**
 1. Project: Q_i, K_i, V_i
 2. Score: $s_j^{(i)} = \langle Q_i, K_j \rangle / \sqrt{k}$
 3. Softmax: $\alpha_j^{(i)}$
 4. Weighted sum: $\text{AttnVec}_i = \sum_j \alpha_j^{(i)} V_j$
 5. Output projection: $\tilde{e}^{(i)} = W_O \text{AttnVec}_i$
- All matrices (W_Q, W_K, W_V, W_O) are learned by backpropagation from a task loss.

Worked Example — Part 1

Sentence: “The central bank raised interest rates amid rising inflation.”

Tokenization (9 word tokens, numbered left to right):

The central bank raised interest rates amid rising inflation
1 2 3 4 5 6 7 8 9

Goal: produce $e'^{(5)}$, the contextualized vector for *interest* — one that encodes how the word is used *in this sentence* (the interest of monetary-policy rates, not “curiosity” or “financial stake”).

- **Attention score** $s_j^{(5)} = \langle Q_5, K_j \rangle / \sqrt{k}$ measures how much *interest* should listen to token j . Suppose, after scaling, the model finds:

Token j	$s_j^{(5)}$
rates ($j = 6$)	2.3
inflation ($j = 9$)	1.9
raised ($j = 4$)	1.1
bank ($j = 3$)	0.5
the ($j = 1$)	-0.2

Intuitively: *interest* is most strongly tied to *rates* and *inflation* — the words that fix its meaning here.

Worked Example — Part 2

- **Softmax** $\rightarrow$ **attention weights**. $\alpha_j^{(5)}$ is the *fraction* of attention *interest* pays to token j — a probability, with $\sum_j \alpha_j^{(5)} = 1$:

$$\alpha_6^{(5)} \approx 0.50, \quad \alpha_9^{(5)} \approx 0.40, \quad \alpha_4^{(5)} \approx 0.05, \quad \dots$$

50 % of the attention goes to *rates*, 40 % to *inflation*, a sliver to everything else.

- **Weighted sum of value vectors** (what the token actually “reads”):

$$\text{AttnVec}_5 \approx 0.50 V_6 + 0.40 V_9 + 0.05 V_4 + \dots$$

- **Project back:** $\tilde{e}^{(5)} = W_O \text{AttnVec}_5$.
- **Result:** $\tilde{e}^{(5)}$ is a vector in $\mathbb{R}^d$ that represents “*interest, in a context dominated by rates and inflation*” — exactly the meaning we want for monetary-policy text.
- The same computation runs in parallel for every position i , producing $\tilde{e}^{(1)}, \dots, \tilde{e}^{(9)}$ — every token gets its own context-aware vector in one step.

How Q , K , V Are Learned

- **What is trainable (the “controls” θ):** in every layer, W_Q, W_K, W_V, W_O , plus the FFN weights W_1, W_2, b_1, b_2 , LayerNorm parameters, the token embedding matrix $\mathcal{E}$, and the output head W_{vocab} . All are initialized randomly and updated by gradient descent on a **single global loss**.
- **Notation:** $t^{(i)}$ is the i -th token of a training sequence s . The sum $\sum_s \sum_i$ runs over every position i in every sequence s in the corpus.

- **Common training objectives:**

- **Autoregressive LM** (GPT-style):

$$\mathcal{L}(\theta) = - \sum_s \sum_i \log P_{\theta}(t_s^{(i+1)} \mid t_s^{(1:i)}).$$

- **Masked LM** (BERT-style): hide tokens at random and predict them from bidirectional context.
- **Classification fine-tuning:** cross-entropy on a sentence-level label.
- **One loss, all parameters.** Backpropagation sends gradients through softmax, dot products, Q/K/V projections, residuals, LayerNorm, *and the FFN* — one backward pass updates every parameter together. There is no separate training signal for attention vs. the feed-forward block.
- Nothing tells the model “attend to neighbors” or “ignore stopwords” — it discovers attention patterns from data.

Multi-Head Attention

- One (Q, K, V) trio captures only one “view” of the sequence. **Multi-head attention** runs H such trios in parallel and concatenates their outputs.
- Each head h has its own projection matrices $W_Q^{(h)}, W_K^{(h)}, W_V^{(h)}$, all of dimension $k = d/H$.
- After running H attention computations independently, concatenate and project:

$$\text{MultiHead}_i = W_O^* \left[\text{AttnVec}_i^{(1)}; \dots; \text{AttnVec}_i^{(H)} \right].$$

- **Do heads have different objectives?** *No — all H heads share the same training loss* (e.g. next-token cross-entropy). What differs is their random initialization. Specialization is **emergent**: under one global loss, H parallel feature detectors diversify because identical copies of the same head would be redundant.
- **Observed specializations in financial text:**
 - Head 1: monetary-policy relations (*inflation* $\leftrightarrow$ *interest*)
 - Head 2: temporal phrases (*Q1, next month, by year-end*)
 - Head 3: hawkish vs. dovish stance markers

Nobody told the heads what to look for — they were not assigned separate objectives.

- Total parameter cost is essentially the same as a single-head attention with the full dimension d

Full Transformer Layer

Residual Connection and Layer Normalization

- After the attention output $\tilde{e}^{(i)}$, the layer combines it with the original input via a *residual connection* and then a *layer normalization*:

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}).$$

- **Why residual?**

- Preserves the original information — attention can only *add* to it, never destroy it
- Provides a clean gradient path back through dozens of layers (the trick that made deep networks trainable)

- **Why layer normalization?**

- Normalizes activations across the feature dimension to mean 0, variance 1
- Adds learnable scale γ and shift β
- Stabilizes optimization dynamics, especially for very deep stacks

Feed-Forward Network (MLP)

- **Division of labor inside a layer.** Attention mixes information *across* tokens; the FFN then transforms *each* token individually given what it just gathered. Attention = the meeting room; FFN = desk work after the meeting.
- **Why it exists.** Attention alone is a linear map of value vectors. The FFN supplies the *non-linear* transformation that lets the network represent arbitrary functions of a token's contextualized embedding. Empirically, FFN weights store most of the model's factual knowledge — delete them and attention-only models lose world facts even if the attention pattern is intact.
- After attention + residual + norm, each position is passed independently through a small feed-forward network:

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2.$$

- **Shape conventions:** $W_1: d \rightarrow 4d$ (expand), activation σ (ReLU / GELU / SwiGLU), $W_2: 4d \rightarrow d$ (contract). The $4d$ expansion is where most of an LLM's parameters live — capacity for richer non-linear transformations.
- Followed by another residual connection and layer normalization:

$$e'^{(i)} = \text{Norm}(u^{(i)} + f^{(i)}).$$

One Layer — Putting It All Together

- A single Transformer layer can be summarized by the flow:

$$e^{(i)} \xrightarrow{\text{Self-Attention}} \tilde{e}^{(i)} \xrightarrow{\text{Res+Norm}} u^{(i)} \xrightarrow{\text{MLP}} f^{(i)} \xrightarrow{\text{Res+Norm}} e'^{(i)}.$$

- Operationally:

$$\tilde{e}^{(i)} = W_O \sum_j \alpha_j^{(i)} V_j,$$

$$u^{(i)} = \text{Norm}(e^{(i)} + \tilde{e}^{(i)}),$$

$$f^{(i)} = W_2 \sigma(W_1 u^{(i)} + b_1) + b_2,$$

$$e'^{(i)} = \text{Norm}(u^{(i)} + f^{(i)}).$$

- Same $(\mathbb{R}^d)^n$ shape in and out: a layer is a structured non-linear update.

After the Transformer: Where We Are

- **What went in:** a sequence of n tokens, converted to embeddings + positional codes, $[e^{(1)}, \dots, e^{(n)}]$ with each $e^{(i)} \in \mathbb{R}^d$.
- **What comes out of the top layer:** n *contextualized* vectors $[e'^{(1)}, \dots, e'^{(n)}]$, same shape, same dimension. Each $e'^{(i)}$ encodes, in its d coordinates,
 - (a) the identity of token $t^{(i)}$,
 - (b) its syntactic / semantic role in *this* sequence,
 - (c) what every other token contributed to its meaning (causally masked in GPT-style models).
- **What we have NOT yet done:** no token generated, no class assigned, no number returned. Contextualized embeddings are an **intermediate data structure**, not the final answer.
- **What comes next:** a small *task head* on top — a vocabulary projection (generation) or a class projection (classification) — turns the embeddings into the output we want. The next slide is that final step.

Stacking L Layers

- The output of one layer becomes the input of the next:

$$[e^{(1)}, \dots, e^{(n)}]_{\text{layer } \ell+1} = [e'^{(1)}, \dots, e'^{(n)}]_{\text{layer } \ell}.$$

- After L layers, each token's vector has accumulated context from the entire sequence (subject to causal masking, in the autoregressive case).
- **Different layers learn different things** (empirically):
 - Lower layers: surface syntax, local agreement, morphology
 - Middle layers: phrase- and clause-level structure
 - Upper layers: discourse, factual associations, task-relevant abstractions
- **What we do with the top layer (“task heads”).**
 - **Generation:** logits = $W_{\text{vocab}} e'^{(n)}$, then $P(\text{next} = w) = \text{softmax}(\text{logits})_w$. Training loss:
 $\mathcal{L} = -\sum_i \log P(t^{(i+1)} \mid t^{(1:i)})$ — the same cross-entropy that shaped every weight in the stack during pretraining.
 - **Classification:** logits = $W_{\text{class}} e'^{([\text{CLS}])}$, then $P(\text{class} = c) = \text{softmax}(\text{logits})_c$. Training loss:
 $\mathcal{L} = -\sum \log P(y_{\text{true}} \mid x)$ on a labeled set (fine-tuned encoders: BERT, CentralBankRoBERTa, ...).

Transformer vs. Word2Vec

Property	Word2Vec	Transformer
Representation	1 static vector / word	contextual vector / token
Architecture	2-layer Skip-gram / CBOW	multi-head attention + MLP, deep stack
Context scope	fixed local window (± 5)	full sequence, every token sees every token
Position info	none	explicit positional encodings
After training	lookup table; model discarded	model <i>is</i> the network used downstream
Typical size	$\sim 10^7$ params (CPU OK)	10^8 – 10^{12} params (GPU required)
Best for	similarity, feature inputs	generation, classification, QA, RAG

The big shift: from a frozen lookup table of meanings to a deep, dynamic model where meaning emerges from interaction with context.

Pretrain, Generate, Fine-tune

Pretraining: Self-Supervised Language Modeling

- **Data:** hundreds of billions to trillions of tokens of natural text and code (web pages, books, papers, code repositories, filings, news, ...).
- **Objectives:**
 - **Causal LM** (GPT family): $\mathcal{L}(\theta) = - \sum_s \sum_i \log P_\theta(t_s^{(i+1)} | t_s^{(1:i)})$. *In words:* at every position in every training sequence, the model outputs a vocabulary distribution; we penalize it by the negative log-probability of the token that *actually* appeared.
 - **Masked LM** (BERT family): hide a random subset of tokens and predict them bidirectionally, $\mathcal{L}(\theta) = - \sum_{i \in \text{masked}} \log P_\theta(t^{(i)} | t^{(\setminus i)})$.
- **This is maximum-likelihood estimation** of a conditional distribution. The novelty vs. classical statistics is the parameterization (10^{11} -parameter neural network) and the scale of the data — not the statistical objective.
- **Optimization:** Adam/AdamW, large batches, warmup + cosine LR, gradient clipping, mixed precision. **Compute:** weeks–months on thousands of accelerators, 10^{23} – 10^{25} FLOPs. No human labels — the next token *is* the label.

Generating the Next Token

- **After pretraining** we have a trained θ . Generation uses that θ to extend a prompt.
- Given a prompt $t^{(1)}, \dots, t^{(n)}$, run the Transformer to obtain $e'^{(n)}$.
- Compute logits and softmax:

$$P(t^{(n+1)} = w \mid t^{(1)}, \dots, t^{(n)}) = \text{softmax}(W_{\text{vocab}} e'^{(n)})_w.$$

- **Pick** a token according to a sampling strategy, **append** it to the context, and **repeat** until a stop token, a length limit, or an end-of-sequence marker.
- **Cost:** naively, generating n tokens is $O(n^2)$ in attention; practical implementations cache keys and values.

Sampling Strategies

- **Greedy:** pick $\arg \max_w P(w \mid \cdot)$. Deterministic, but often repetitive and bland.
- **Top- k sampling:** restrict to the k most likely tokens, renormalize, then sample. Cuts off the long tail of low-probability nonsense.
- **Top- p (nucleus) sampling:** restrict to the smallest set of tokens whose cumulative probability exceeds p (e.g. 0.9). Adapts to how peaked the distribution is.
- **Temperature τ :** rescale logits by $1/\tau$ before softmax.
 - $\tau \rightarrow 0$: deterministic, like greedy
 - $\tau = 1$: model's natural distribution
 - $\tau > 1$: more diverse, more creative, more risky
- **For empirical research, set $\tau = 0$** (and report it). Reproducibility is more valuable than creativity when measuring something.

Fine-Tuning for Economics

- A pretrained LLM has *general* linguistic knowledge. Economic tasks often benefit from a small, targeted update.
- **Three common strategies:**
 - **Full fine-tuning:** update every parameter. Highest quality, highest cost. Risks overwriting general knowledge with limited task data.
 - **Adapters / LoRA** (Low-Rank Adaptation): freeze the original weight W and learn a low-rank update AB^T with $A \in \mathbb{R}^{d \times r}$, $B \in \mathbb{R}^{k \times r}$, $r \ll \min(d, k)$. Trains $\sim 0.1\%$ of the parameters at near full-FT quality. *Preferred* for most economics applications.
 - **Domain Adaptive Pretraining (DAPT):** continue pretraining on a domain corpus (FOMC statements, MD&A texts, central bank speeches) before fine-tuning on the labeled task.
- **Examples:** sentiment classification on FOMC statements, 10-K segment labeling, NER for corporate filings, summarization of monetary policy reports.

Fine-Tuning: LoRA Mechanics

- **Idea:** instead of updating every weight matrix W in the model, freeze W and learn a *low-rank correction*:

$$W_{\text{eff}} = W + AB^{\top}, \quad A \in \mathbb{R}^{d \times r}, \quad B \in \mathbb{R}^{k \times r}, \quad r \ll \min(d, k).$$

Only A and B are trainable during fine-tuning; the original W stays fixed.

- **Parameter savings** (LLaMA-7B):
 - Full fine-tuning updates $\sim 7 \times 10^9$ parameters.
 - LoRA with $r = 8$ on attention projections updates $\sim 4 \times 10^6$ parameters ($\approx 0.06\%$ of the model).
 - Near-equivalent task quality for most classification, sentiment, and style tasks.
- **Why it matters for economists:** a 7B or even 70B base model can be adapted on a single GPU or a modest cloud instance — no industrial-scale hardware required.
- **Inference:** after training, merge the update back into the weights ($W \leftarrow W + AB^{\top}$); deployment cost is identical to the base model.
- **Tooling:** HuggingFace `peft` wraps any pretrained model with LoRA adapters in a few lines of code.

Fine-Tuning: Workflow for an Economics Task

Running example: classify FOMC statements as hawkish / dovish / neutral.

- **Dataset:** 500–2,000 hand-coded statements (e.g. Hansen–Kazinnik, 2023). Split by *meeting date* — train on 1994–2015, validate on 2016–2019, test on 2020–2024 — to avoid leaking future regimes into training.
- **Base model:** an encoder (BERT / RoBERTa) with a classification head is usually enough and cheaper than decoder fine-tuning for three-way classification.
- **Task formatting:** prepend [CLS], tokenize and pad to a fixed length, attach a softmax head over 3 classes to the [CLS] output vector.
- **Hyperparameters (LoRA defaults):** learning rate $1\text{--}3 \times 10^{-4}$, rank $r = 8$, batch size 16–32, 3–5 epochs, early stopping on validation macro-F1.
- **Evaluation:** report macro-F1 and a per-class confusion matrix; hand-audit 20–30 misclassifications — most errors reveal label noise or systematic biases worth fixing before scaling.
- **Reference template:** CentralBankRoBERTa (see Case Study 3a) follows exactly this recipe, with a DAPT pass on the FOMC corpus preceding the labeled fine-tuning.

Transformer vs. LDA Topic Models

LDA / topic models

- Document = mixture of latent topics
- Bag of words: ignores order
- Output: topic-word and document-topic distributions
- Strength: interpretable themes
- Weakness: cannot generate coherent prose

Transformers

- Token-by-token, order-aware
- Captures syntax and long-range dependencies
- Output: contextual embeddings + a generative model
- Strength: rich generation, fluent prose, classification
- Weakness: less directly interpretable

When to use what: LDA is still excellent for thematic structure and quick descriptive analysis on small corpora. Transformers dominate when you need generation, classification, or context-aware embeddings.

Three Ways to Adapt an LLM

Method	What changes	Best for
Prompt engineering	Only the input text	Quick experiments, zero-cost adaptation
Fine-tuning (LoRA / DAPT)	Some model weights	New behavior or style on a fixed domain
Retrieval-Augmented Generation (RAG)	Relevant external context injected at inference time	New <i>facts</i> the model never saw

- All three are non-exclusive. In practice you often combine them.
- RAG addresses the problem we will dwell on at the end of this lecture: *the model's knowledge has a cutoff date*.
- Lecture 8 is dedicated to RAG.

Revisiting: Why One Equation Enables So Much

- Now that we understand pretraining, fine-tuning, and generation, the opening claim makes mechanical sense:

$$p(t^{(n+1)} \mid t^{(1)}, \dots, t^{(n)}) \quad \text{is all there is.}$$

- **Summarizing a 10-K filing:** the prompt carries the document and the instruction; the model predicts summary tokens one by one because pretraining exposed it to millions of (long-document, summary) pairs.
- **Extracting FOMC stance:** fine-tuning on labeled central bank text shifts the next-token distribution so that stance-label tokens are most probable after a monetary-policy sentence.
- **Generating scenario forecasts:** condition on macro variables and a scenario description; next-token prediction fills in the narrative consistently with the conditioning.
- **Writing and debugging code:** the training corpus included billions of code tokens; the objective is identical — next token, given context.
- **The unifying insight:** all tasks are the same computation. Pretraining provides the knowledge; prompts and fine-tuning steer *which* next-token distribution we sample from. Every capability is a special case of the one equation.

Economic Applications

Three Featured Case Studies

1. **ECB Word2Prices** (Araujo, Bokan, Comazzi & Lenza, 2025)
 - Word2Vec on ECB statements + BVAR for inflation forecasting
2. **Patents and Worker Exposure** (Kogan et al., 2019)
 - TF-IDF + cosine similarity between patents and occupational tasks
3. **Central Bank Sentiment with BERT** (Pfeifer & Marohl, 2023; Ganguli et al., 2024)
 - Fine-tuned RoBERTa for sentiment toward different economic agents
 - PaECTER patent embeddings for innovation dynamics

After the three deep dives, we briefly tour the wider “application zoo” and the validation pitfalls econometricians need to keep in mind.

Case 1 — ECB Word2Prices: Setup

Araujo, Bokan, Comazzi & Lenza (2025), ECB WP No. 3047

- **Question:** can embeddings of ECB communications improve euro-area inflation forecasting?
- **Data:**
 - ECB press conference introductory statements, 2002Q1–2023Q4
 - Target: HICP_x (core inflation), monthly → quarterly
- **Method:**
 - For each quarter q , retrain Word2Vec on the corpus available *up to* q (no look-ahead)
 - Average word vectors per statement; aggregate to a quarterly text vector
 - Stack text vector with macro variables; fit a Bayesian VAR
 - Forecast 1–4 quarters ahead
- **Why this design?** Re-training quarterly avoids the look-ahead bias that plagues most LLM-based forecast exercises, while remaining computationally cheap.

Case 1 — Python Skeleton

```
# Quarterly Word2Vec retraining for ECB inflation forecasting
from gensim.models import Word2Vec
from statsmodels.tsa.api import VAR
import numpy as np

forecasts = []
for q in quarters:
    # 1. Build corpus available at quarter q (no look-ahead)
    corpus_q = [s for s in all_statements if s.date <= q]
    # 2. Train Word2Vec on the available corpus
    w2v = Word2Vec(corpus_q, vector_size=100, window=5,
                   sg=0, min_count=2, epochs=20)
    # 3. Average word vectors per statement, then per quarter
    stmt_vec = [np.mean([w2v.wv[w] for w in s if w in w2v.wv], axis=0)
                 for s in corpus_q]
    text_q = aggregate_quarterly(stmt_vec, dates)
    # 4. Stack with macro variables and fit a BVAR
    Y = np.column_stack([macro_q, text_q])
    bvar = VAR(Y).fit(maxlags=4)
    # 5. Forecast 1-4 quarters ahead
    forecasts.append(bvar.forecast(Y, steps=4))
```

Case 1 — Results and Lessons

- **Forecast performance:**

- BVAR with Word2Vec text features outperforms a baseline AR / BVAR without text at horizons of 1–4 quarters
- Sentiment dictionaries (e.g. Loughran-McDonald) deliver only marginal gains relative to the no-text baseline
- Heavy LLMs (BERT, fine-tuned) can match or beat Word2Vec but are much more expensive and harder to retrain quarterly — look-ahead bias becomes a real risk

- **Take-aways for empirical economists:**

- “Lighter is better” if it lets you respect the timing structure of the forecasting exercise
 - Frequently-retrained Word2Vec is a strong baseline before reaching for an LLM
 - Always document: training corpus cutoff at every step, vector dimension, window, retraining frequency
- Replicable framework with zero GPU requirements — ideal for grad-student empirical work.

Case 2 — Patent Text and Worker Exposure

Kogan, Papanikolaou, Schmidt, Seegmiller (2019, 2023)

- **Question:** how does exposure to technological innovation affect workers' earnings and employment?
- **Method:**
 - Compute TF-IDF vectors for U.S. patent texts and for O*NET occupational task descriptions
 - Measure cosine similarity between each occupation and each patent
 - Aggregate up: occupation-level technological exposure
- **Findings:**
 - High-exposure occupations experience earnings declines and reduced employment stability
 - Effects are present for both low- and high-wage workers
- **Why this is in our lecture:** a textbook example of how a *simple* method (TF-IDF + cosine) on a *thoughtfully chosen* corpus can answer a substantive economic question. You don't need transformers to make a contribution, but you do need a clean identification strategy.

Case 3a — CentralBankRoBERTa

Pfeifer & Marohl (2023), Journal of Finance and Data Science

- **Goal:** extract *agent-specific* sentiments from central bank communications — separating views toward households, firms, and the financial sector.
- **Method:**
 - Fine-tune RoBERTa (a BERT variant) on labeled central bank statement data
 - For each sentence, output a sentiment vector indexed by economic agent
- **Findings:**
 - Agent-specific sentiments predict regional inflation and employment indicators
 - Substantially more informative than overall “hawkish vs. dovish” scores
- **Why this works:** the fine-tuned transformer captures *which subject* is being discussed, not just the polarity of the language. That kind of structured extraction is hard for both bag-of-words and zero-shot LLM prompting.

Case 3b — PaECTER for Patent Innovation

Ganguli, Lin & Reynolds (2024), NBER WP

- **Goal:** measure innovation dynamics from patent text using transformer-based embeddings.
- **Model: PaECTER** (Patent Embeddings from Citation-Enhanced Transformer Representations)
 - Initialize from BERT-base
 - Fine-tune on patent corpora
 - Use citation pairs as a supervised signal during fine-tuning (related patents pulled together in embedding space)
- **Findings:**
 - Significantly more accurate similarity judgments than TF-IDF or LDA
 - Distinguishes incremental from radical innovations
 - Long-run trend: patents are increasingly diverging in semantic space (broader exploration)
- **Take-away:** fine-tuned transformer embeddings give a richer, citation-aware view of technological evolution than purely text-based or count-based methods.

Frontier (2024–25): LLMs for Central Bank Communication

- **CB-LMs** (Gambacorta et al., BIS WP 1215, 2024)
 - Domain-adapted encoder retrained on 37,000 central bank papers + 18,000 speeches
 - Outperforms generic BERT on FOMC stance classification
 - But: GPT-4 wins on complex narrative tasks — no single model dominates
- **GPT-4 for FedSpeak** (Hansen & Kazinnik, 2023)
 - GPT-4 classifies FOMC statement sentences at *expert-level* accuracy
 - Outperforms fine-tuned BERT/RoBERTa; provides human-consistent explanations
 - Zero-shot performance collapses domain-adaptation barrier for many tasks
- **GPT-4o on 25,000 FOMC Sentences** (Federal Reserve FEDS Note, December 2024)
 - Topic classification across 2010–2024 minutes at scale
 - Detects “financial stability” discourse spikes in 2020 and 2023 banking crises
 - LLMs reveal *thematic shifts* that sentiment scores miss
- **74,882 Central Bank Documents** (Silva, Moriya & Veyrune, IMF WP 2025/109)
 - 169 central banks, 1884–2025; LLM-coded stance + directional communication index
 - Forecasts future policy rate changes; documents shift to forward-looking language

Frontier (2024–25): LLMs in Asset Pricing

- **Expected Returns and LLMs** (Chen, Kelly & Xiu, NBER WP 31502, 2024)
 - LLM embeddings of financial news $\Rightarrow$ cross-sectional equity return prediction
 - Substantially outperforms TF-IDF and momentum; gains largest on *negation* and complex narratives
 - Evidence of slow price adjustment to information that only LLM embeddings extract
- **AI Asset Pricing Models** (Kelly, Kuznetsov, Malamud & Xu, NBER WP 33351, 2025)
 - Transformer architecture embedded directly in the stochastic discount factor M_{t+1}
 - Multi-head attention learns cross-asset dependencies from 132 stock characteristics
 - Large reductions in pricing errors vs. previous ML factor models
- **Asset Embeddings** (Gabaix, Koijen, Richmond & Yogo, NBER WP 33651, 2025)
 - Embeddings learned from *investor portfolio holdings* via skip-gram (same math as Word2Vec)
 - Arithmetic: `tech + growth - hardware` $\approx$ `software firms`
 - GPT-4 generates economic narratives for firm clusters in the learned embedding space
- **Key insight:** the same transformer and embedding toolkit that encodes language can encode financial structure — text and quantitative data are no longer separate worlds.

The Application Zoo

A non-exhaustive list of text-and-LLM applications in economics:

- **Baker, Bloom & Davis (2016)** — newspaper-keyword Economic Policy Uncertainty index
- **Hoberg & Phillips (2016)** — TF-IDF cosine similarity over 10-K business descriptions to map product-market competition
- **Hassan & Hollander et al. (2019)** — bigram-based corporate political risk, predicting firm investment
- **Li, Mai, Shen & Yan (2021)** — Word2Vec to measure five corporate culture dimensions from earnings calls
- **Lopez-Lira & Tang (2023)** — ChatGPT zero-shot sentiment on news headlines, predicting next-day stock returns
- **Hartley (2025)** — LLM-driven reconstruction of EPU on 19th-century newspapers (robust to OCR errors)
- **BIS (2025)** — structured LLM sentiment as inputs to MIDAS / DFM nowcasting

Validation Caveats for Econometricians

- **Cosine similarity is not innocent.** Three common confounds inflate similarity between firm/policy texts:
 - **Industry jargon:** firms in the same SIC code share boilerplate language
 - **Templates:** regulatory filings re-use chunks of text verbatim
 - **Length:** longer documents have smoother averages and look more similar
- **Standard tools for robustness:**
 - Permutation tests — shuffle labels within an industry, check if observed similarity exceeds the within-industry baseline
 - Residual embeddings — subtract the industry mean before measuring similarity
 - Bootstrap the entire pipeline (text $\rightarrow$ embedding $\rightarrow$ measurement $\rightarrow$ regression) for honest standard errors
- **Dictionary methods are not innocent either.** The Loughran-McDonald (2011) finance-specific dictionary corrects mismatches in the older Harvard IV-4 list (e.g. “liability” is not negative in financial accounting).
- **Always validate** a sample of model outputs against human coding before scaling up.

A Six-Step Workflow for LLM-Assisted Measurement

1. **Pre-alignment.** Hand-code 100–200 examples; iterate on the prompt or fine-tuning recipe until you reach acceptable agreement.
2. **Full annotation.** Run the model on the entire sample with a frozen prompt and a frozen model version.
3. **Gold-standard audit.** Blind-code 10% of the sample and compute Cohen's κ (publication threshold $\kappa \geq 0.75$).
4. **Category-level reporting.** Don't just report overall accuracy; show the confusion matrix by class. Minority classes are often where models fail silently.
5. **External validity check.** Link the text-derived measure to a real economic outcome (returns, investment, employment) before treating it as a measurement.
6. **Disclosure.** Report prompt, model version, temperature ($\tau = 0$), date, all preprocessing, and provide code. Reproducibility is essential when the model behind the API can change without notice.

Limitations

The Knowledge Cutoff

- Every pretrained LLM is trained on a *static snapshot* of text. After the snapshot ends, the model learns nothing new.
- Each model has a **knowledge cutoff date**. After this date:
 - Events are invisible to the model
 - New papers, datasets, policy announcements simply don't exist as far as it is concerned
 - Asking “what happened in March 2026?” against a model trained through 2024 will at best yield “I don't know” — and at worst, a confident fabrication
- **Why not just retrain?**
 - Cost: \$10M–\$100M+ for a frontier model
 - Time: weeks to months
 - Energy and infrastructure constraints
 - Practically impossible to keep “current”
- This is not a bug — it is structural. Any solution that needs *fresh* information must work *around* the cutoff, not against it. **This is the core motivation for Lecture 8.**

Hallucination Risk

What it looks like

- Fabricated citations to papers that do not exist
- Made-up statistics that look plausible
- Non-existent laws, cases, article numbers
- Subtly wrong details in otherwise correct prose

Mitigations

- Require and verify citations externally
- Use retrieval (Lec08) to ground answers in real documents
- Use tools / code: numbers from a calculator
- Validate a sample by hand
- Set temperature $\tau = 0$ when measuring

Why does it happen? The model is trained to produce *plausible-sounding* next tokens, not *true* ones. Truth is a property of the world; plausibility is a property of language.

Context Window Limits

- Even modern models with 100,000+ token context cannot swallow arbitrarily large datasets:
 - A 200-page 10-K filing easily exceeds short context windows
 - Decades of FOMC transcripts cannot be loaded at once
 - Multi-firm or multi-country panels of textual data certainly cannot
- **Practical consequences:**
 - Long inputs must be *chunked*, which can break coherence and cross-reference
 - Important information may sit outside the window altogether
 - Attention cost is $O(n^2)$ in the context length, so doubling n quadruples compute and memory
- Newer architectures (sparse attention, sliding-window attention, RoPE-extension, state-space hybrids) push the limit but do not eliminate it.
- **Implication:** you cannot just dump “all the data” into a prompt. You need a way to *retrieve* the relevant pieces and feed them in — another argument for RAG.

Prompt Sensitivity and External Validity

- **Primacy bias** (Brucks & Toubia, 2025): on a balanced 50/50 binary task, GPT-4 picks the *first* option 64%–92% of the time. The fix is to randomize option order across runs and average.
- **Prompt phrasing**: small wording changes can flip a classification. Robust analyses report results under multiple paraphrases.
- **“Parrot psychology”**: apparent “preferences” or “beliefs” may simply reflect text patterns memorized from training data, not genuine reasoning.
- **WEIRD bias**: the training corpus is disproportionately Western, English, and Internet-native. Behavioral simulations of, say, rural Chinese households inherit that bias.
- **Model version drift**: closed-source models can change between API calls. Reproducibility requires you to freeze the model version, document the date, and ideally cross-check against an open-source baseline.
- **Take-away**: treat the prompt as part of your research design, not as a transparent interface. Report it.

Transition to RAG

The Brain Metaphor

An LLM is like a very capable brain that finished school on a fixed date.

What the brain knows

- General language and reasoning
- Broad economics, math, programming
- Published papers up to its cutoff
- Common patterns, idioms, code recipes

What the brain does not know

- Anything that happened after the cutoff
- Your private FOMC corpus
- A specific firm's internal documents
- This morning's data release
- Your co-author's unpublished draft

Question: how do we let this brain access the information on the right *without* retraining it?

Next Step: Extending the Brain

- **Three options for adding new knowledge to an LLM:**
 1. **Retrain** from scratch with the new data. Effective, but expensive (\$M–\$B), slow (months), impractical for individual researchers.
 2. **Fine-tune** on the new data. Cheaper, but unreliable for injecting new *facts* — fine-tuning is good at changing style and behavior, less so at memorizing isolated facts.
 3. **Retrieve** relevant pieces at inference time and inject them into the prompt. Cheap, dynamic, and the model never has to be touched.
- **Lecture 8** is dedicated to option 3: **Retrieval-Augmented Generation (RAG)**. A frozen LLM combined with a queryable external store of documents.
- **Lecture 9** goes one step further — **agentic AI**, where the brain not only retrieves information but also *acts*: runs code, fetches URLs, writes files, calls other tools. Claude Code is the running example.
- **Lecture 10** brings it all together with case studies of how economists actually use these tools today.

Summary

1. **Inputs:** tokenization + embeddings + positional encodings turn text into a sequence of vectors.
2. **Self-attention** is the core operation of the Transformer: Q, K, V projections $\rightarrow$ scaled dot-product scores $\rightarrow$ softmax $\rightarrow$ weighted sum of values.
3. **Stacking** layers (residuals, normalization, feed-forward networks) yields contextualized representations.
4. **Generation** is autoregressive: project the last embedding to the vocabulary, sample, append, repeat.
5. **Established applications:** TF-IDF + cosine (Kogan et al.), Word2Vec + BVAR (ECB Word2Prices, Araujo et al.), fine-tuned RoBERTa (CentralBankRoBERTa, Pfeifer & Marohl).
6. **2024–25 frontier:** GPT-4 for Fedspeak (Hansen & Kazinnik); LLM embeddings for cross-sectional returns (Chen, Kelly & Xiu); transformer in SDF (Kelly et al.); asset embeddings from holdings (Gabaix et al.).
7. **LLMs have a fixed knowledge cutoff.** Next: extend the brain with retrieval — **Lecture 8: RAG.**